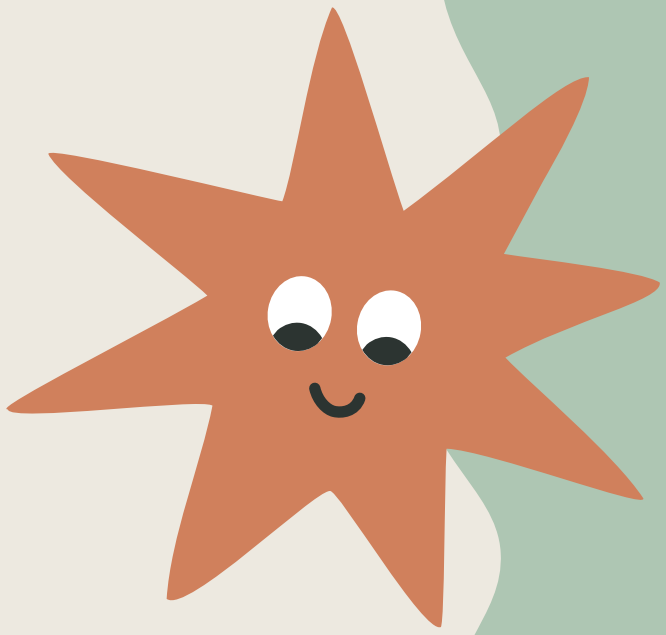
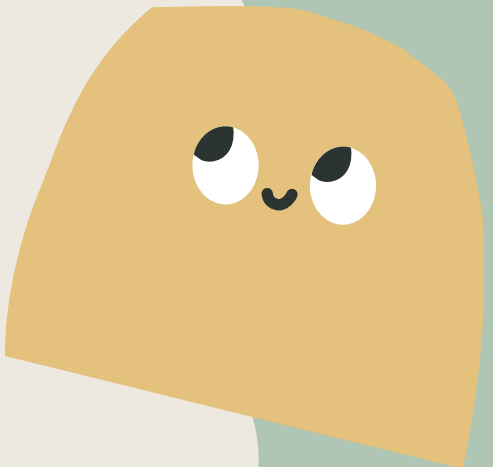
Dictionaries in Python

Lecture 9





Learning Objectives

➢By the end of this lecture, students should be able to:

- Understand **key-value pairs**
- Create dictionaries
- Access dictionary values
- Update existing values
- Delete dictionary elements
- Use important dictionary methods
- Loop through dictionaries
- Work with nested dictionaries
- Apply dictionaries to real-world and AI-related problems

Prepared By: Ehtisham Rasheed

What is a Dictionary?

- A **dictionary** is a Python data structure that stores data in **key-value pairs**.
- Think of it like a real dictionary:
  - Word → Meaning
- In Python:
  - Key → Value
- Syntax & Example:

```
dictionary = {
    key1: value1,
    key2: value2,
    key3: value3
}
```

```
student = {
    "name": "Ali",
    "age": 20,
    "gpa": 3.45
}
```

Prepared By: Ehtisham Rasheed

Creating a Dictionary

```
student = {
    "name": "Ali",
    "age": 20,
    "department": "Computer Science",
    "gpa": 3.45
}

print(student)
```

➢Output:

```
{'name': 'Ali', 'age': 20, 'department': 'Computer Science', 'gpa': 3.45}
```

```
print(type(student))
```

```
<class 'dict'>
```

Prepared by: Ehtesham Rasheed

Accessing Dictionary Values

- Unlike lists and tuples, we normally access dictionary data using **keys**.

-

```
student = {
    "name": "Ali",
    "age": 20,
    "gpa": 3.45
}

print(student["name"])
print(student["age"])
print(student["gpa"])
```

Output

```
Ali
20
3.45
```

Prof. Ehtisham Rasheed

Keys Must Be Unique

- A dictionary cannot have two different values associated with the same key.
- For example:

```
student = {
    "name": "Ali",
    "name": "Ahmed"
}

print(student)
```

- Python keeps the last value

```
{'name': 'Ahmed'}
```

Prepared By Tisham Rasmed

Adding a New Key-Value Pair

➢Suppose we have

```
student = {
    "name": "Ali",
    "age": 20
}
```

➢We can add a new item

```
student["gpa"] = 3.45

print(student)
```

➢Result

```
{'name': 'Ali', 'age': 20, 'gpa': 3.45}
```

Prepared by Ehtisham Rasheed

Updating a Dictionary Value

➢Suppose:

```
student = {
    "name": "Ali",
    "age": 20,
    "gpa": 3.45
}
```

➢We want to change GPA:

```
student["gpa"] = 3.75

print(student)
```

➢Output:

```
{'name': 'Ali', 'age': 20, 'gpa': 3.75}
```

Important rule

If the key exists → **update**
If the key doesn't exist → **add**

Prepared By: Ehtisham Rasheed

Deleting Dictionary Elements

➢There are several ways.

➢**del**

```
student = {
    "name": "Ali",
    "age": 20,
    "gpa": 3.45
}

del student["age"]

print(student)
```

pop() removes a key and returns its value.

```
student = {
    "name": "Ali",
    "age": 20,
    "gpa": 3.45
}

removed_value = student.pop("age")

print("Removed:", removed_value)
print(student)
```

Prepared By: Eng. Rasheed

Deleting Dictionary Elements

- **popitem()** removes the last inserted key-value pair.

```
student = {
    "name": "Ali",
    "age": 20,
    "gpa": 3.45
}

item = student.popitem()

print("Removed:", item)
print(student)
```

- **clear()** removes everything

```
student = {
    "name": "Ali",
    "age": 20
}

student.clear()

print(student)
```

Output

```
{}
```

Prepared By: Ehtisham Rasheed

Important Dictionary Methods

Method	Purpose
`keys()`	Get all keys
`values()`	Get all values
`items()`	Get key-value pairs
`get()`	Safely access a value
`update()`	Add/update multiple items
`pop()`	Remove a specific item
`popitem()`	Remove last item
`clear()`	Remove everything
`copy()`	Create a copy

keys(), values(), and items()

```
student = {"name": "Ali", "age": 20, "gpa": 3.45}
print("Keys:", student.keys())
print("Values:", student.values())
print("Items:", student.items())
print("Keys as list:", list(student.keys()))
```

➢Output:

```
Keys: dict_keys(['name', 'age', 'gpa'])
Values: dict_values(['Ali', 20, 3.45])
Items: dict_items([('name', 'Ali'), ('age', 20), ('gpa', 3.45)])
Keys as list: ['name', 'age', 'gpa']
```

get()

- Consider:

```
student = {
    "name": "Ali",
    "age": 20
}
```

- This works: `print(student["name"])`
- But `print(student["gpa"])` causes **KeyError**
- Instead, we can use: `print(student.get("gpa"))`
- Output:

```
None
```

- We can also provide a default value: `print(student.get("gpa", "Not Available"))`
- Output:

```
Not Available
```

Prepared By Ehtisham Rasheed

update()

- We can update multiple values at once.
-

```
student = {
    "name": "Ali",
    "age": 20
}

student.update({
    "age": 21,
    "gpa": 3.75
})

print(student)
```

Output

```
{'name': 'Ali', 'age': 21, 'gpa': 3.75}
```

Prepared by: Engineer Ibtisham Rasheed

Checking Whether a Key Exists

➢Use **in**

```
student = {
    "name": "Ali",
    "age": 20,
    "gpa": 3.45
}

print("name" in student)
print("email" in student)
```

➢Output:

```
True
False
```

Prepared By: Ehtisham Rasheed

Looping Through a Dictionary

Loop through keys

```
student = {
    "name": "Ali",
    "age": 20,
    "gpa": 3.45
}

for key in student:
    print(key)
```

Loop Through Values

```
for value in student.values():
    print(value)
```

Loop Through Keys and Values

```
for key, value in student.items():
    print(key, ":", value)
```

Prepared By: Ebtesham Rasheed

Nested Dictionaries

- A dictionary can contain another dictionary.
- This is called a **nested dictionary**.
- Example:

```
students = {
    "student1": {
        "name": "Ali",
        "age": 20,
        "gpa": 3.45
    },

    "student2": {
        "name": "Sara",
        "age": 21,
        "gpa": 3.75
    }
}
```

```
print(students["student1"]["name"])
```

```
print(students["student2"]["gpa"])
```

Ehtisham Rasheed

In-Class Practice

- **Exercise 1 — Student Dictionary**
- Create a dictionary containing:
  - name
  - age
  - department
  - semester
  - GPA
- Print each value.
- **Exercise 2 — Update Student**
- Change the student's GPA and add an email address.

Prepared By: Ehtisham Rasheed

In-Class Practice

➢Exercise 3 — Marks Dictionary

➢Create:

```
marks = {
    "Python": 85,
    "AI": 91,
    "DSA": 88,
    "Database": 78
}
```

➢Calculate:

- total marks
- average
- highest marks
- lowest marks

Prepared By: Ehtisham Rasheed